

Deviation lookup by indicator – fix

A saved deviation does not come back after reopening the Health page, and is inserted again on the next Apply. Seven changes across five layers. The foreign key to `indicator_configuration` stays as Benoit asked; only the lookup changes.

What is happening

The log shows `AircraftConfiguration ... action=CREATION` on every Apply. Each Apply creates a new configuration version, with new `indicator_configuration` rows and new ids.

#	What happens
1	The row loads carrying version 1's <code>indicator_configuration</code> id
2	Apply saves the deviation with version 1's id – and creates version 2
3	On reopen, the lookup asks for version 2's ids
4	The deviation still points at version 1, so nothing matches: deviation is null, the row is not green, <code>deviationId</code> stays null
5	The next Apply sees <code>deviationId</code> null and inserts the same deviation again

The green shown right after Apply is only the in-memory flag set by `isValidData`. It was never loaded from the database, which is why it disappears when you change tab and come back.

Confirm before changing anything. Run this in pgAdmin before and after one Apply. If the count grows, this is the cause.

```
SELECT count(*) FROM public.indicator_configuration;
```

The fix: match deviations through the indicator, whose id does not change between versions, instead of through the configuration row, whose id does.

Server

Step 1 — IndicatorDeviation – replace both named queries

`mgs-jpa-model/.../datamodel/health/monitoring/IndicatorDeviation.java`

Replace the two existing `@NamedQuery` annotations above the class. The query now walks from the deviation to its configuration row to the indicator.

```
@NamedQuery(name = "findActiveDeviationsByIndicators",
    query = "SELECT d FROM IndicatorDeviation d "
        + "WHERE d.indicatorConfiguration.indicator.id IN :indicatorIds "
        + "AND d.endDate IS NULL")
@NamedQuery(name = "findDeviationHistoryByIndicators",
    query = "SELECT d FROM IndicatorDeviation d "
        + "WHERE d.indicatorConfiguration.indicator.id IN :indicatorIds "
        + "AND d.endDate IS NOT NULL")
```

Step 2 — IndicatorDeviationDao – match the new query names

`mgs-services-impl/.../service/health/common/ejb/IndicatorDeviationDao.java`

Replace both finders. The query name string must match Step 1 character for character; a mismatch fails at runtime, not at compile time.

```

public List<IndicatorDeviation> findActive(List<String> indicatorIds) {
    if (indicatorIds.isEmpty()) {
        return new ArrayList<>();
    }
    TypedQuery<IndicatorDeviation> q = manager.createNamedQuery(
        "findActiveDeviationsByIndicators", IndicatorDeviation.class);
    q.setParameter("indicatorIds", indicatorIds);
    return q.getResultList();
}

public List<IndicatorDeviation> findHistory(List<String> indicatorIds) {
    if (indicatorIds.isEmpty()) {
        return new ArrayList<>();
    }
    TypedQuery<IndicatorDeviation> q = manager.createNamedQuery(
        "findDeviationHistoryByIndicators", IndicatorDeviation.class);
    q.setParameter("indicatorIds", indicatorIds);
    return q.getResultList();
}

```

Step 3 — IHealthCommonService and HealthCommonService – no change

Both methods already take a List<String> and pass it straight through. The list now carries indicator ids instead of configuration ids, but the type is the same, so nothing here needs editing.

Step 4 — IndicatorDeviationDto – add one field

mgs-services-interfaces/.../service/health/common/dto/IndicatorDeviationDto.java

After the indicatorConfigurationId field. The client needs the indicator id to match a deviation to its row.

```

private String indicatorId;

public String getIndicatorId() {
    return indicatorId;
}

public void setIndicatorId(String indicatorId) {
    this.indicatorId = indicatorId;
}

```

Step 5 — HealthCommonBusiness – collect indicator ids and fill the new field

mgs-health-business/.../business/health/impl/HealthCommonBusiness.java

Three edits in this one class.

a. Replace the private helper getIndicatorConfigurationIds with this one:

```

private List<String> getIndicatorIds(String aircraftId) throws EntityNotFoundException {
    List<String> ids = new ArrayList<>();
    HealthMonitoringConfiguration configuration = getLatestConfiguration(aircraftId);
    for (HealthMechanicalComponentConfiguration mcc :
        configuration.getMechanicalComponentConfigurations().values()) {
        for (IndicatorConfiguration ic : mcc.getIndicatorConfigurations().values()) {
            ids.add(ic.getIndicator().getId());
        }
    }
    return ids;
}

```

b. In findActiveDeviations and findDeviationHistory, call the new helper:

```

return toDtos(healthCommonService.findActiveDeviations(getIndicatorIds(aircraftId)));

return toDtos(healthCommonService.findDeviationHistory(getIndicatorIds(aircraftId)));

```

c. In toDtos, directly after dto.setIndicatorConfigurationId(...), add:

```

dto.setIndicatorId(d.getIndicatorConfiguration().getIndicator().getId());

```

toDtos runs on the server with the session open, so reaching through the lazy indicatorConfiguration to its indicator is safe here.

Client

Step 6 — HealthAlarmsPreferenceStore → createAllMechanicalComponents – key by indicator

mgs-health-app/.../healthalarms/store/HealthAlarmsPreferenceStore.java

In the loop that builds deviationMap, change the key.

```
// BEFORE
deviationMap.put(dto.getIndicatorConfigurationId() + "|" + dto.getSeverity(), dto);

// AFTER
deviationMap.put(dto.getIndicatorId() + "|" + dto.getSeverity(), dto);
```

Step 7 — HealthAlarmsPreferenceStore → getThresholdModels – look up by indicator

In the loop that sets the law and the baseline, change the lookup to match.

```
// BEFORE
IndicatorDeviationDto deviation = deviationMap.get(
    icId + "|" + thresholdModel.getSeverity().name());

// AFTER
IndicatorDeviationDto deviation = deviationMap.get(
    indicator.getId() + "|" + thresholdModel.getSeverity().name());
```

Leave `setIndicatorConfigurationId(icId)` exactly as it is. The save still needs the configuration row id for the foreign key; only the lookup moves to the indicator.

Build and test

#	Action
1	Delete the duplicate rows already in the table, keeping one per indicator and severity
2	Rebuild in order: mgs-jpa-model, mgs-services-interfaces, mgs-services-impl, mgs-health-business, then the client
3	Redeploy the EAR
4	Edit row A, type a TE, Apply, OK – expect one row in indicator_deviation
5	Change tab and come back – row A is still green and not editable
6	Edit row B, type a TE, Apply, OK – expect exactly two rows in total
7	Close the Parameter window, reopen it – both rows still green

Existing rows need no migration. They point at old configuration rows, but those rows still reference the same indicator, so the new lookup finds them without any data change.

One thing to know

New deviations are saved against the configuration row that was loaded, which is the version before the Apply that created it. With the lookup going through the indicator this makes no difference to what the user sees. It does mean the foreign key records which configuration version the deviation departed from, rather than the one that was current afterwards – worth mentioning to Benoit, since that is arguably the more useful thing to record anyway.